
Testare automata bazata pe modele

JSXM – Framework Java de testare bazat pe Stream

X-Machines

Alexe Vasile-Paul, grupa 505 (master BDTS)

Nițoi Antonio, grupa 506 (master IS)

Prezentare generala a Model-Based Testing si Stream X-Machines

De ce testare bazata pe modele?

Probleme in testarea clasica:

- Teste scrise manual
- Acoperire incompleta
- Erori greu de anticipat

Solutia: **Model-Based Testing (MBT)**

Ce sunt Stream X-Machines?

Stream X-Machines sunt modele formale utilizate pentru a ilustra functionalitatile sistemelor software prin reprezentarea grafica a acestora ca o multime de stari si tranzitii.

Astfel, SXM-urile se bazeaza pe automatele finite, pe care le extind cu noi functionalitati precum memoria interna care le permite sa stocheze date relevante pentru functionalitatea aplicatiei.

De asemenea, SXMs sunt Turing complete, astfel fiind capabile sa simuleze orice sistem software.

Din ce este alcatuit un Stream X-Machine

Componentele unui Stream X-Machine (SXM) sunt similare cu cele ale unui automat finit.

- **Stari:** reprezinta diferite configuratii in care se poate afla sistemul.
- **Input-uri:** date pe care le primeste dintr-un mediu extern.
- **Functie de tranzitie:** o functie care ne spune cum sa trecem dintr-o stare in alta.

In plus, un SXM mai este alcatuit si din:

- **Output-uri:** raspunsul SXM-ului.
 - **Memorie:** stocare interna pentru stocarea datelor relevante procesarii.
-

La ce se folosesc Stream X-Machines

- Stream X Machines sunt utile pentru a simplifica sisteme complexe in scopul testarii si ajuta la o testare completa a sistemelor software prin folosirea lor pentru generarea de teste.
 - SXMs ofera o reprezentare mai simpla a unui sistem, punand accentul pe starile, tranzitiile, si memoria folosita de sistem. Astfel, fac simpla testarea logicii sistemului, dar pot sa nu detecteze probleme legate de elemente non-deterministe precum interactiunea cu utilizatorii sau dependentele externe.
 - Prin reprezentarea tuturor starilor posibile ale unui sistem, SXMs permit de asemenea testarea tuturor combinatiilor posibile de input-uri, ceea ce scade sansa de a rata anumite edge cases.
 - De asemenea, izoland logica sistemului, SXMs fac mai simpla testarea functionalitatilor de baza, fara a mai intra in detalii in metoda de implementare a acestora.
-

Beneficiile testarii folosind Stream X-Machines

- **Abordare sistematica si exhaustiva:**
Permite testarea tuturor tranzitiilor intre stari posibile si tuturor combinatiilor de input.
 - **Concentrare asupra functionalitatilor de baza:**
Izoleaza logica sistemului pentru a o testa.
 - **Descoperirea erorilor devreme:**
Se observa defecte in gandire/logica inca din stadiul de development.
 - **Sustine restul testelor:**
Asigura o baza pentru proiectarea unei suite de teste mai amanuntite.
-

Limitările testării cu Stream X-Machines

- **Complexitatea sistemelor din viața reală**
Modelarea trebuie efectuată manual, astfel este atât posibil să fie ratate cazuri, dar necesită și foarte multă muncă.
- **Se concentrează pe logică**
SXM caută greșeli în logica aplicației, astfel nu este în stare să detecteze erorile cauzate de dependențele externe sau interacțiunea cu utilizatorul.

În concluzie, SXMs ajută la testarea sistemelor, dar este recomandată să fie folosite împreună cu alte metode de testare pentru a asigura o testare comprehensivă.

Prezentare a framework-ului JSXM

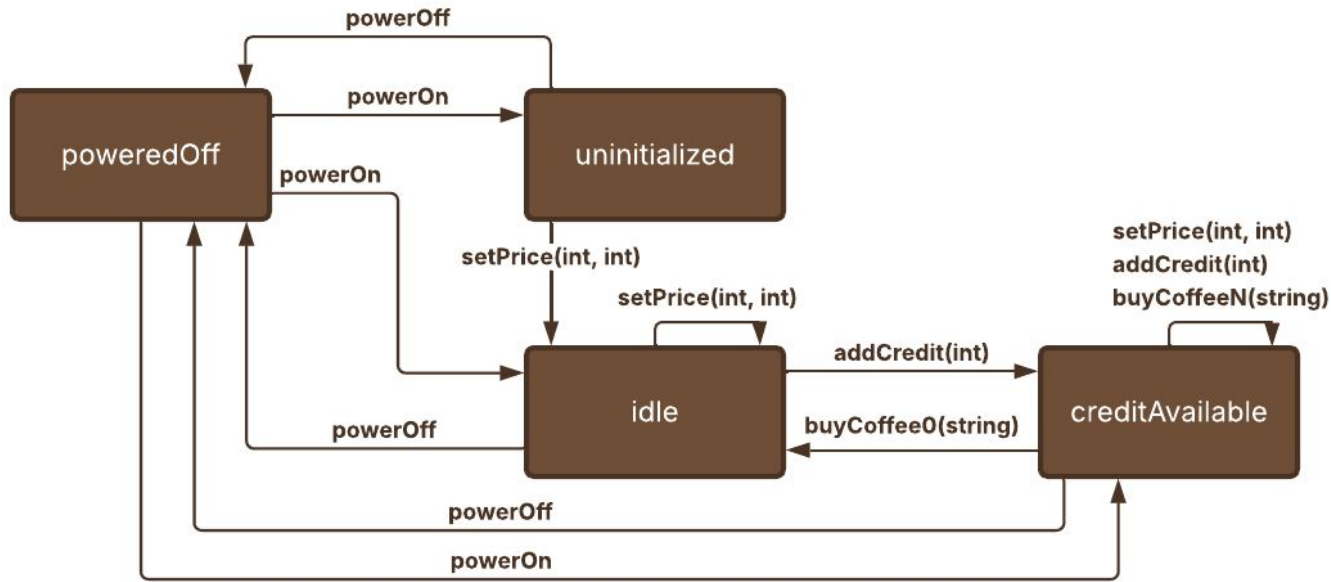
Ce este JSXM

JSXM este un framework Java pentru Model-Based Testing ce faciliteaza crearea si utilizarea de SXMs pentru testare, oferind urmatoarele facilitati:

- Definirea modelului folosind fisiere de tip XML
 - Animarea modelului implementat
 - Generarea automata de teste pe baza arhitecturii SXM-ului
 - Implementarea automata a acestor teste folosind JUnit
 - Rularea automata a acestor teste si oferirea unui raport cu numarul de teste pe care implementarea aplicatiei l-a trecut
-

Un exemplu de testare folosind JSXM

Coffee Machine



Apoi a urmat implementarea SXM-ului si a adaptorului necesar pentru a traduce input-urile SXM-ului in apeluri de functii corespunzatoare pentru modelul implementat in Java si valorile/exceptiile rezultate din rularea metodelor in output-uri sau fault-uri din SXM.

Am folosit functionalitatea animate-gui pentru a ne asigura ca SXM-ul corespunde functionalitatii pe care ne-o doream de la modelul nostru si pentru a ne asigura ca SXM-ul este complet.

Am folosit apoi JSXM pentru a genera o suita de 2204 de teste unitare si pentru a le rula, obtinand urmatorul rezultat:

```
Tests run: 2204, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:  
0.366 sec
```

Live Demo

Va multumim pentru atentie
